

Data Classification Task Report

목차:

1.서론

2.EDA

3.Logistic Regression

4.Support Vector Machine

5.Decision Tree & Random Forest

6.K-Nearest Neighbors

7.결론

1.서론

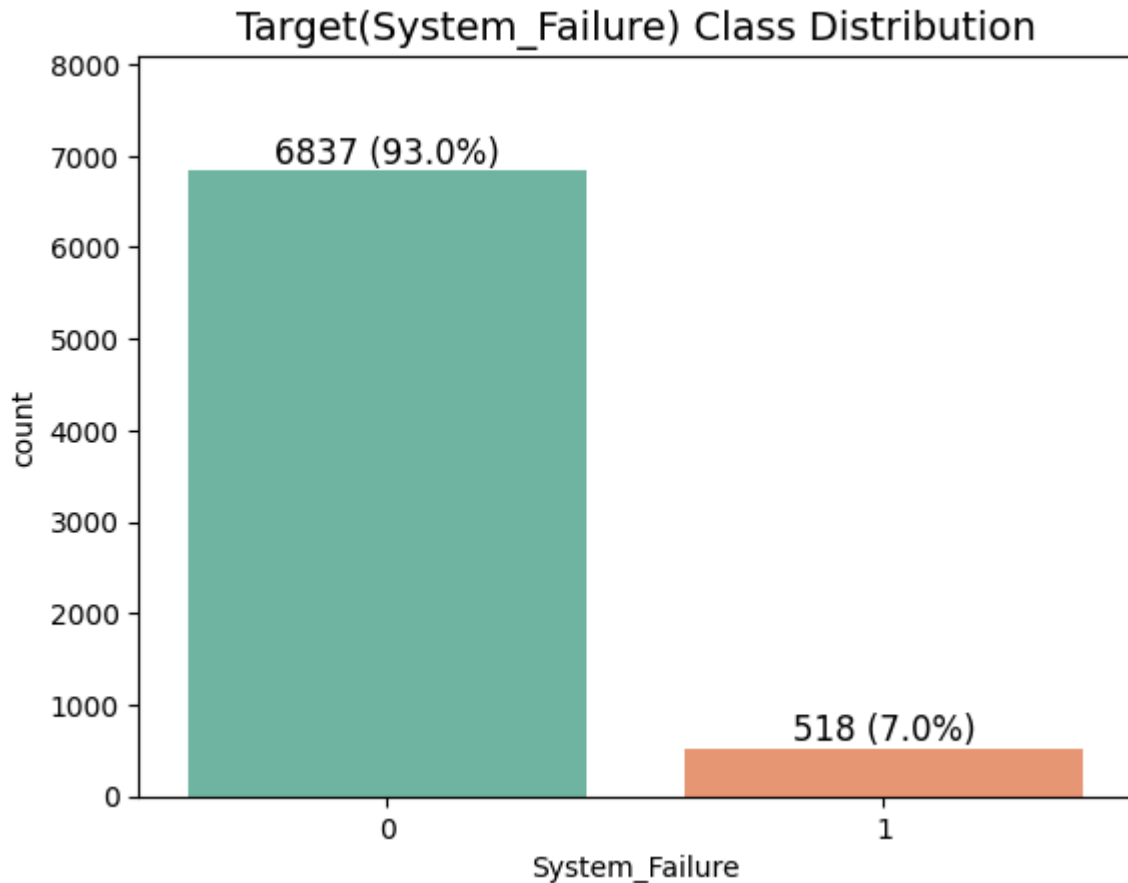
UR3 Cobotops 머신러닝 프로젝트 중 Classification을 실행한다.

주제는 로봇팔의 Robot ProtectiveStop와 grip lost를 종합하여 0과 1의 이산적인 System

Failure 피처를 만들어 각 관절들의 Speed, Temperature, Current 피처로 분류해내는것이다.

사용되는 모델은 Logistic Regression, Support Vector Machine, Decision Tree, Random Forest, K-Nearest Neighbors이다.

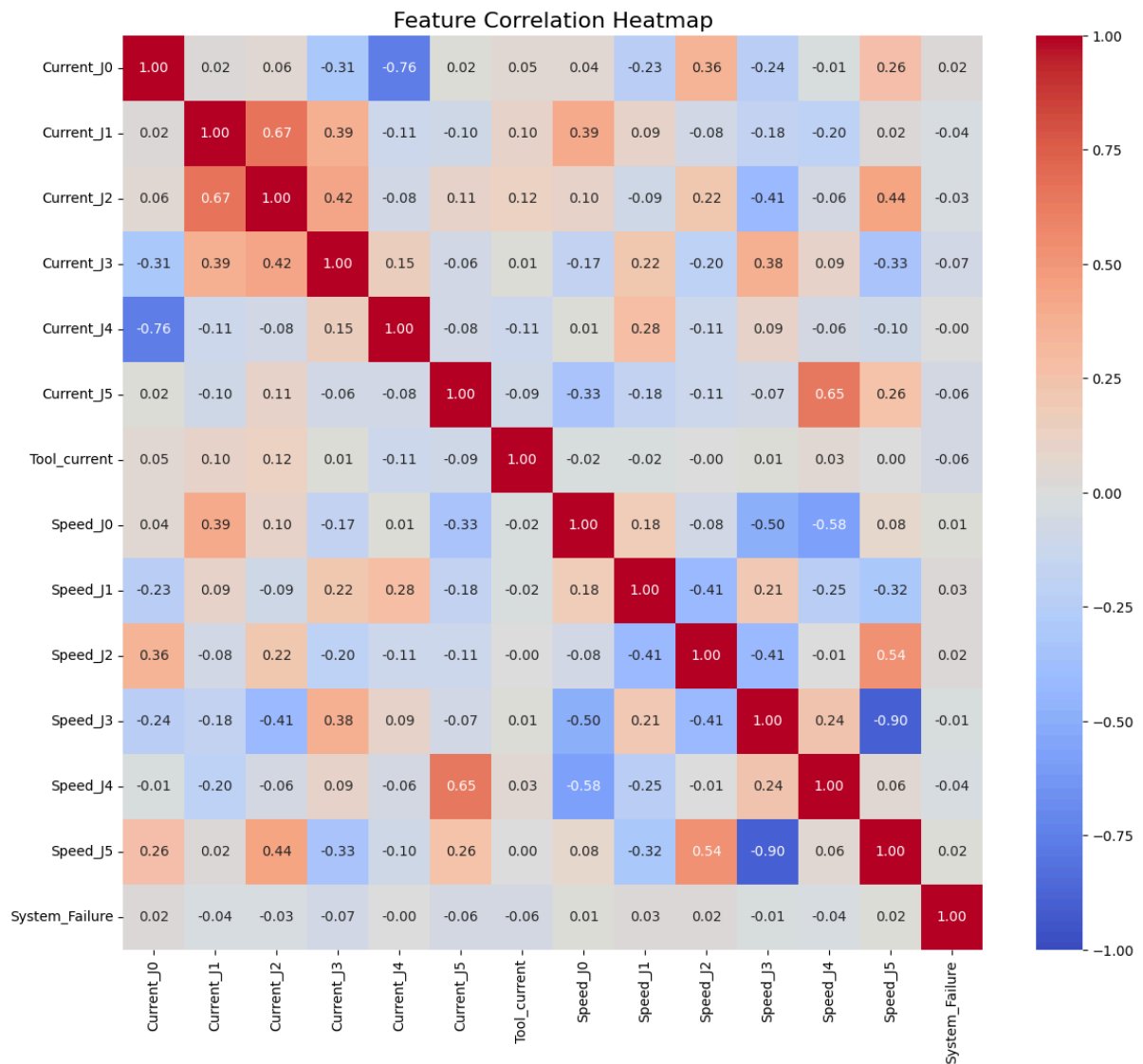
2.EDA



Robot ProtectiveStop와 grip lost 중 어떤것이라도 발생했다면 비정상 시스템 정지로 판단하는 **System Failure** 라벨을 만들고 클래스 분포를 보았다.

정상이 93%이고 비정상이 7%인 클래스 불균형을 보인다.

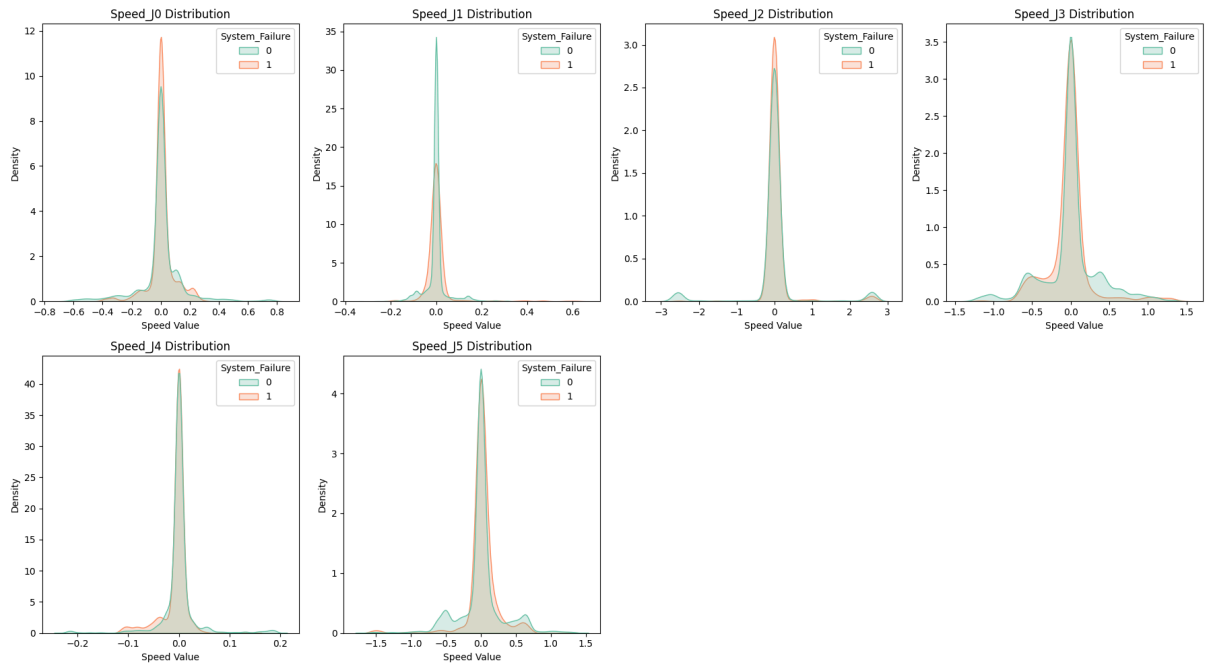
그대로 학습시킨다면 모든 상황을 정상으로만 예측해도 **Accuracy**가 93%를 도달하는 정확도의 역설을 일으키므로, **SMOTE**같은 클래스 불균형을 해소하는 방법을 사용해 고장 데이터를 얼마나 잘 잡았는지 평가하는 **F1-Score**나 **Recall**, **ROC-AUC**를 반드시 확인해야함을 시사한다.



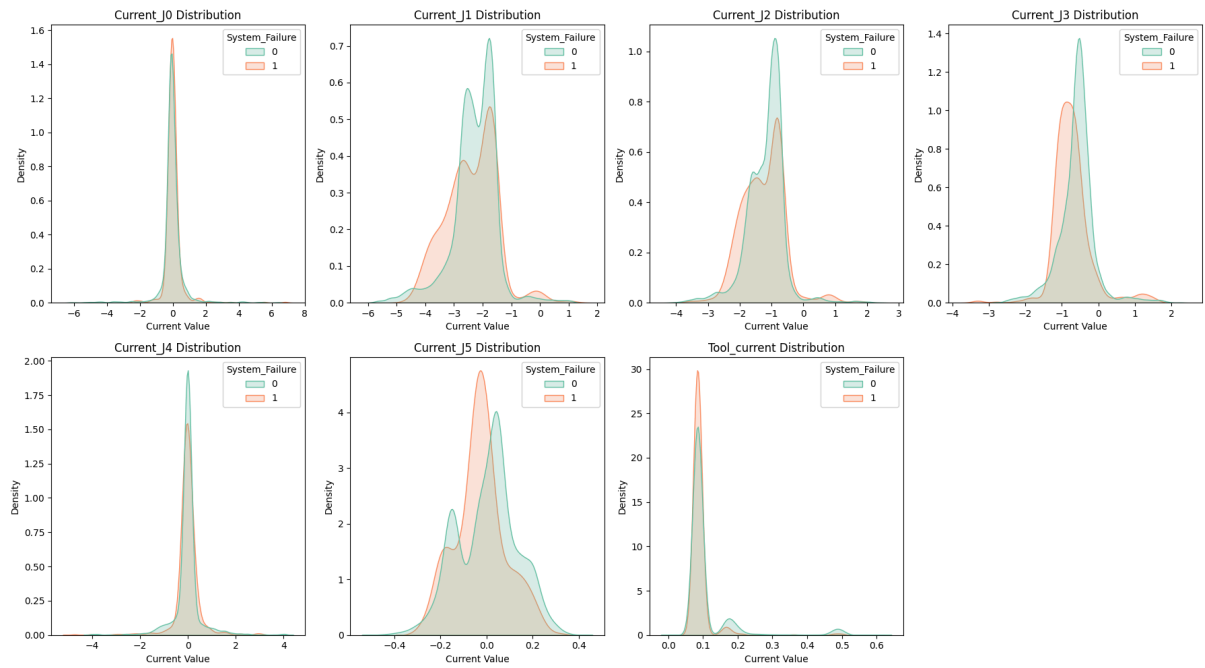
히트맵을 봤을때, 타겟 변수인 **System Failure**과 다른 피쳐들간의 관계가 매우 희미하다.

이는 선형적 관계가 없다는 것을 의미하며 전류, 속도 등이 높아지거나 낮아진다고 해서
고장 확률이 선형적으로 곧바로 증가하거나 감소하지 않는다.

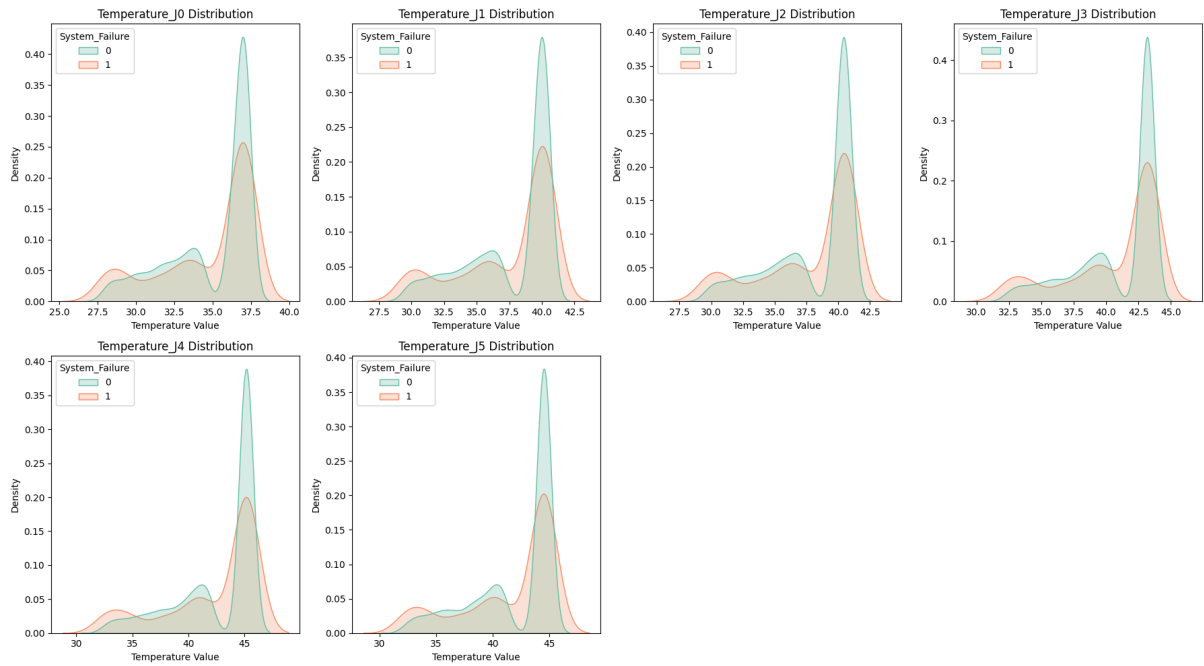
따라서 로지스틱 회귀나 **SVM** 같은 모델들보다 트리 기반 앙상블 모델이 더욱 좋은 성능을
보일것이라 예측된다.



관절별 속도와 고장 여부의 관계를 보면, 정상 분포와 고장 분포가 대부분 겹쳐 있어, 속도 값 자체만으로는 고장 여부를 전혀 판별할 수 없다.



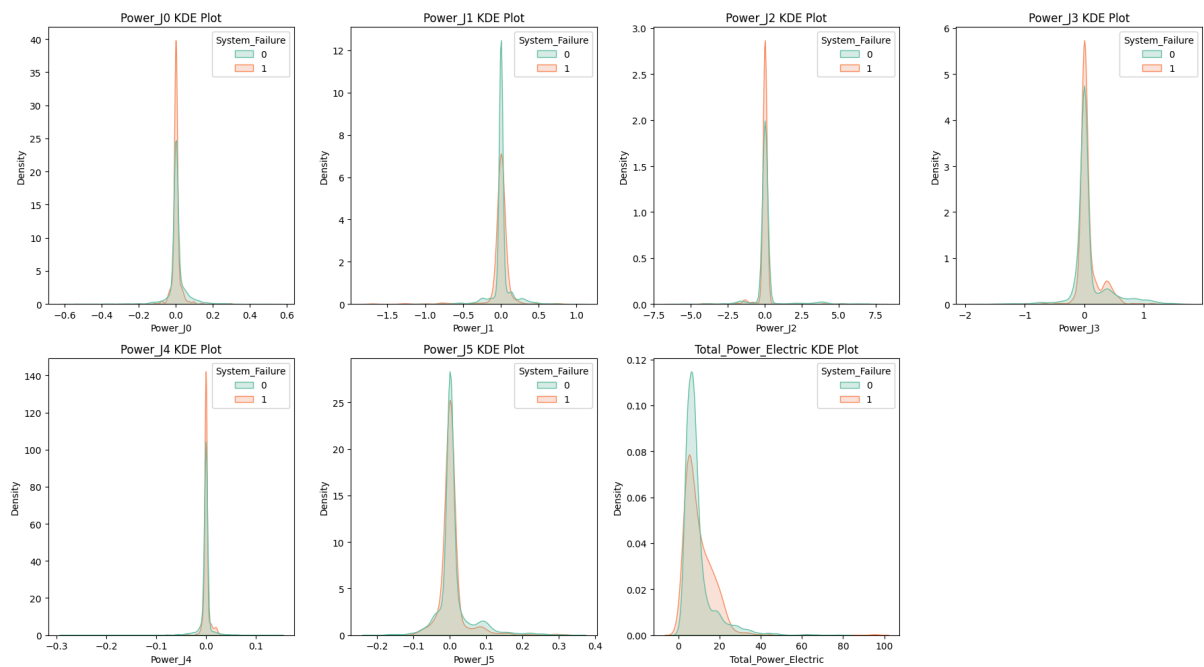
전류를 보면, 평균값은 비슷하지만, 고장 났을 때 데이터가 미세하게 더 넓게 퍼지거나 꼬리가 두꺼워진다.



온도를 보면 정상과 고장 상태일 때 데이터가 밀집되는 온도 구간이 확실하게 다르다.

그럼 물리 기반 파생 변수인 **Power**를 만들어서 보자.

$\text{Power} = \text{Current} \times \text{Speed}$ 는 속도가 빠르는데 전류도 높게 먹는다면 심한 과부하 상태일 수 있다. 반대로, 속도는 0인데 전류가 높다면 힘을 억지로 주고 있는 고장 전조증상일 수 있다.



Power J0 ~ Power J5는 정상일때와 고장일때의 봉우리의 높이가 다르다. 총 전력량을 보면
앞선 그래프들과 다르게 오른쪽으로 고장 그래프가 좀 튀어나와있는걸 확인할 수 있었다.

3. Logistic Regression

가장 단순하고 해석이 직관적인 로지스틱 회귀로 시작한다.

```
df.columns = df.columns.str.strip()

# 2. 결측치 없는 클린 데이터 분리
df_clean = df.dropna().copy()
df.rename(columns={'Temperature_T0': 'Temperature_J0'}, inplace=True)

# 3. 파생 변수 생성 (Power = Current * Speed 등)
current_cols = [f'Current_J{i}' for i in range(6)]
speed_cols = [f'Speed_J{i}' for i in range(6)]
temp_cols = [f'Temperature_J{i}' for i in range(6)]
for i in range(6):
    df_clean[f'Power_J{i}'] = df_clean[f'Current_J{i}'] * df_clean[f'Speed_J{i}']

df_clean['Total_Power_Electric'] = (df_clean[current_cols]**2).sum(axis=1)

# 4. 입력 피쳐(x) 설정
input_cols = current_cols + speed_cols + temp_cols + ['Tool_current'] + [f'Power_J{i}' for i in range(6)] + ['Total_Power_Electric']
X = df_clean[input_cols]

print(f"데이터 크기: {df_clean.shape}")
print(f"피쳐 개수: {len(input_cols)}개")
✓ 0.0s
```

데이터 크기: (7355, 31)
피쳐 개수: 26개

온도와 전류, 속도, 파생변수 모두 넣고 학습을 진행한다.

```
# 통합 이진 타겟(System_Failure) 생성: Protective Stop 또는 Grip Lost 발생 시 1
df_clean['Robot_ProtectiveStop'] = df_clean['Robot_ProtectiveStop'].astype(bool).astype(int)
df_clean['grip_lost'] = df_clean['grip_lost'].astype(bool).astype(int)

y = ((df_clean['Robot_ProtectiveStop'] == 1) | (df_clean['grip_lost'] == 1)).astype(int)

print("분류된 타겟(y)의 클래스별 데이터 개수:")
print(y.value_counts().sort_index())
# 0: Normal(정상), 1: Failure(고장)
✓ 0.0s
```

분류된 타겟(y)의 클래스별 데이터 개수:

0	6837
1	518

Name: count, dtype: int64

Protective Stop 또는 Grip Lost 발생 시 1, 아니면 0으로 이진 타겟 라벨을 만든다.

```

# 1. Stratified K-Fold 설정
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# 2. Imblearn Pipeline 구축 (Fold 내에서 SMOTE 적용 -> Logistic Regression 학습)
pipeline = ImbPipeline([
    ('smote', SMOTE(random_state=42)),
    ('lr', LogisticRegression(random_state=42, class_weight='balanced', max_iter=1000))
])

# 3. 튜닝할 파라미터 그리드 (Pipeline 사용 시 모델명__파라미터명 형태로 지정)
param_grid = {
    'lr__C': [0.01, 0.1, 1, 10, 100],
    'lr__penalty': ['l1', 'l2'],
    'lr__solver': ['saga']
}

# 4. GridSearchCV 실행 (데이터 불균형을 고려해 scoring을 'f1_macro'로 설정)
print("--- Logistic Regression 하이퍼파라미터 탐색 및 5-Fold 교차 검증 진행 중 ---")
grid_search = GridSearchCV(
    estimator=pipeline,
    param_grid=param_grid,
    cv=skf,
    scoring='f1_macro',
    n_jobs=-1,
    verbose=1
)

grid_search.fit(X_train_scaled, y_train)

# 5. 최적 파라미터 및 결과 확인
print("\n[GridSearchCV 결과]")
print("[최적의 하이퍼파라미터 설정]")
for param, value in grid_search.best_params_.items():
    print(f"    • {param.replace('lr__', '')}: {value}")
print("")
print(f"최고 교차 검증 F1-Macro 점수: {grid_search.best_score_:.4f}")

```

✓ 14.4s

SMOTE를 적용해 오버샘플링을 진행하고 하이퍼파라미터 튜닝을 진행한다. C는 규제 강도의 역수이고 penalty는 L1, L2 방식의 규제 방식을 정한다.

--- [모델 비교] 상위 10개 Logistic Regression 성능 종합 ---									
	순위	CV F1	Train F1	Test F1	과적합 Gap	Train Acc	Test Acc	Test ROC-AUC	파라미터 설정
0	1	0.4819	0.4942	0.5049	-0.0108	0.642930	0.649898	0.770230	C=10, P=I2, S=saga
1	2	0.4818	0.4925	0.5045	-0.0120	0.640721	0.649218	0.770532	C=1, P=I1, S=saga
2	3	0.4810	0.4948	0.5082	-0.0134	0.644290	0.653297	0.770145	C=100, P=I1, S=saga
3	4	0.4809	0.4945	0.5082	-0.0137	0.643950	0.653297	0.770068	C=100, P=I2, S=saga
4	5	0.4809	0.4947	0.5082	-0.0135	0.644120	0.653297	0.770272	C=10, P=I1, S=saga
5	6	0.4774	0.4887	0.5069	-0.0181	0.636472	0.656016	0.767099	C=1, P=I2, S=saga
6	7	0.4655	0.4806	0.4932	-0.0126	0.623555	0.634942	0.768886	C=0.1, P=I1, S=saga
7	8	0.4639	0.4779	0.4910	-0.0131	0.619646	0.631543	0.765278	C=0.1, P=I2, S=saga
8	9	0.4549	0.4709	0.4823	-0.0114	0.606050	0.617947	0.762872	C=0.01, P=I2, S=saga
9	10	0.4504	0.4570	0.4791	-0.0221	0.588375	0.608430	0.749226	C=0.01, P=I1, S=saga

상위 10개의 모델 결과이다. 모든 결과들의 과적합 gap이 거의 없는 수준이며 오히려 Test F1의 값이 더 높다.

모든 모델의 Test F1은 0.5수준이며 Accuracy는 0.65 수준임을 확인 할 수 있다.

모델이 양성과 음성을 서로 얼마나 잘 구별하는지 나타내는 ROC-AUC는 0.77 수준에

위치하여 모델이 정상과 고장을 어느 정도 구분할 줄은 안다는 뜻이다.

```

[선택된 1위 모델의 하이퍼파라미터]
• C: 10
• penalty: l2
• solver: saga

[Train 데이터 평가]
Accuracy (정확도) : 0.6365
F1-Macro Score    : 0.488730
ROC-AUC Score     : 0.715016

[Train 상세 분류 리포트]

```

	precision	recall	f1-score	support
0 (Normal)	0.97	0.63	0.76	5470
1 (Failure)	0.13	0.70	0.21	414
accuracy			0.64	5884
macro avg	0.55	0.67	0.49	5884
weighted avg	0.91	0.64	0.72	5884

```

[Test 데이터 평가]
Accuracy (정확도) : 0.6560
F1-Macro Score    : 0.506860
ROC-AUC Score     : 0.767099

[Test 상세 분류 리포트]

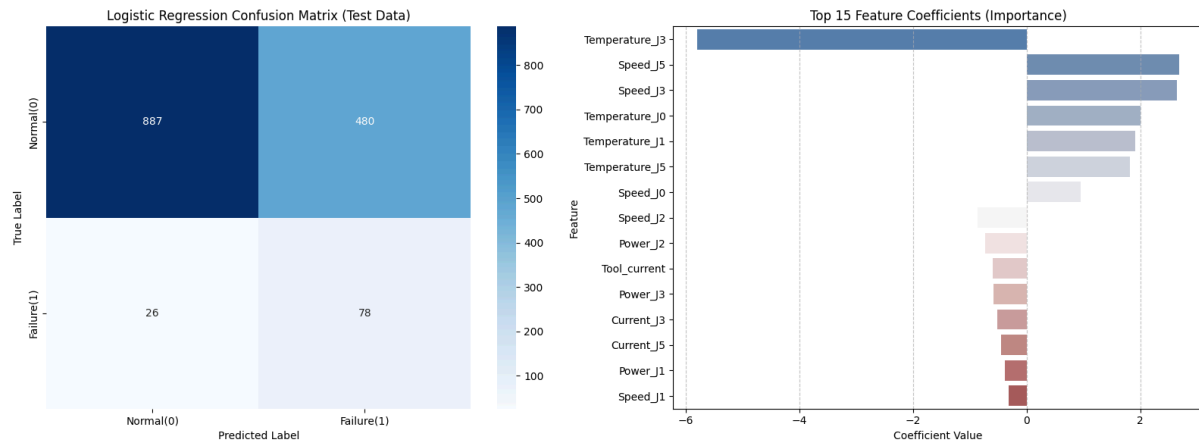
```

	precision	recall	f1-score	support
0 (Normal)	0.97	0.65	0.78	1367
1 (Failure)	0.14	0.75	0.24	104
accuracy			0.66	1471
macro avg	0.56	0.70	0.51	1471
weighted avg	0.91	0.66	0.74	1471

정상 데이터에서 Precision이 0.97로 모델이 정상이라고 판단한 것 중 97%가 진짜 정상였고, Recall이 0.65으로 실제 정상인 데이터 중 65%만 정상으로 찾아냈다. 고장 데이터에서 Precision은 0.14로 모델이 고장이라고 예측한 것 중 실제 고장은 14%뿐이었다. 너무 많은 정상 데이터를 고장으로 오해하고 있다. Recall은 0.75로 실제 고장 난 데이터 중 75%는 고장이라고 찾아냈다.

Accuracy는 65% 정도이지만 데이터가 불균형 상태이므로 신뢰하기 어려운 지표이다.

모델의 균형 잡힌 분류 성능을 보여주는 **F1-Macro Score**도 0.5068으로 현재 매우 낮은 수준이다.



Confusion Matrix를 그려본 결과,

실제 정상(0)을 정상(0)으로 예측 (**True Negative**): 887건

실제 정상(0)을 고장(1)으로 예측 (**False Positive**): 480건

실제 고장(1)을 고장(1)으로 예측 (**True Positive**): 78건

실제 고장(1)을 정상(0)으로 예측 (**False Negative**): 26건

으로 나왔다.

원래 고장 데이터가 극도로 적은 불균형 데이터였기에, 이를 해결하기 위해 **SMOTE**로 가짜 고장 데이터를 많이 생성하였다.

모델이 판단 기준을 대폭 낮춰서 조금만 이상해도 고장으로 분류하는 모델이 되어버린 것으로 추측할 수 있다.

또한 정상과 고장 데이터가 선으로 깔끔하게 나뉘지 않고 서로 섞여 있기 때문에 직선 하나만 그을 수 있는 로지스틱 회귀로는 정상 데이터 영역에 수많은 고장 예측 경계가 침범할 수밖에 없다.

변수의 중요도를 분석한 결과로 **Temperature J3** 피처의 온도가 높을수록 오히려 고장 확률이 낮아진다고 보고있다.

Speed J5, Speed J3, Temperature J0, Temperature J1 등은 양 방향으로 뻗어있어 해당 속도나 온도가 올라갈수록 모델이 고장이라고 판단한다.

```
=====
--- [L1 규제 분석] 2위 모델의 특성 선택(Feature Selection) 결과 ---
=====

[선택된 2위 모델의 하이퍼파라미터]
  • C: 1
  • penalty: l1
  • solver: saga

🔥 [삭제된 피처] 가중치가 정확히 0이 된 피처: 0개
  - 없음 (모든 피처가 사용되었습니다)

☀️ [살아남은 피처] 가중치가 0이 아닌 피처: 26개 (절대값 크기순 정렬)
  - Temperature_J3      : -10.9191
  - Temperature_J4      : 3.3544
  - Temperature_J1      : 3.2728
  - Speed_J5            : 2.8367
  - Speed_J3            : 2.8211
  - Temperature_J0      : 2.2671
  - Temperature_J5      : 2.0206
  - Speed_J0            : 1.0265
  - Speed_J2            : -0.8672
  - Power_J2            : -0.7284
  - Tool_current        : -0.5914
  ...
  - Current_J4          : 0.0751
  - Current_J1          : 0.0501
  - Total_Power_Electric : -0.0436
  - Temperature_J2      : 0.0000
```

L1으로 규제를 가한 2위와 3위 등의 모델이 어떤 피처를 지웠는지 확인한 결과, 모델들은 모든 피처를 사용하고 0으로 만든 피처는 아무것도 없다.

어째서 이런 결과가 나왔는지는 모델의 순위를 보면 알 수 있다.

상위권 모델의 C는 1 아니면 100으로 규제의 강도가 약했다. 이 말은 특정 변수를 0으로 만들면 성능이 더 떨어졌기에 모든 변수의 미세한 패턴을 다 사용해야 성능이 가장 높게 나오기 때문이라고 할 수 있다.

```
# ENN (Edited Nearest Neighbours) 추가 импорт
from imblearn.under_sampling import EditedNearestNeighbours

# 1. Stratified K-Fold 설정
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# 2. Imblearn Pipeline 구축 (Data Leakage 방지 및 복합 샘플링)
# 흐름: SMOTE(오버샘플링) -> ENN(노이즈 언더샘플링) -> Logistic Regression
pipeline = ImbPipeline([
    ('smote', SMOTE(random_state=42)),
    ('enn', EditedNearestNeighbours(n_neighbors=3, kind_sel='all')), # ENN 언더샘플링 추가
    ('lr', LogisticRegression(random_state=42, class_weight='balanced', max_iter=1000))
])

# 3. 튜닝할 파라미터 그리드
# Pipeline 사용 시 모델명__파라미터명 형태로 지정합니다.
param_grid = {
    'lr_C': [0.01, 0.1, 1, 10, 100],
    'lr_penalty': ['l1', 'l2'],
    'lr_solver': ['saga']
}

# 4. GridSearchCV 실행
print("--- [SMOTE + 언더샘플링] Logistic Regression 하이퍼파라미터 탐색 및 5-Fold 교차 검증 진행 중 ---")
grid_search = GridSearchCV(
    estimator=pipeline,
    param_grid=param_grid,
    cv=skf,
    scoring='f1_macro',
    n_jobs=-1,
    verbose=1
)

grid_search.fit(X_train_scaled, y_train)

# 5. 최적 파라미터 및 결과 확인
print("\n[GridSearchCV 결과]")
print("[최적의 하이퍼파라미터 설정]")
for param, value in grid_search.best_params_.items():
    print(f"    {param.replace('lr_', '')}: {value}")
print("")
print(f"최고 교차 검증 F1-Macro 점수: {grid_search.best_score :.4f}")
```

SMOTE는 가상의 데이터를 생성하는 과정에서 소수 클래스 주변에 있는 다수 클래스의 영역까지 침범하여 데이터를 생성하는 한계가 존재한다.

이로 인해 클래스 간의 경계선이 모호해지고, 데이터 내에 노이즈가 발생하여 모델이 과적합되거나 분류 경계를 오인하는 부작용이 나타날 수 있다.

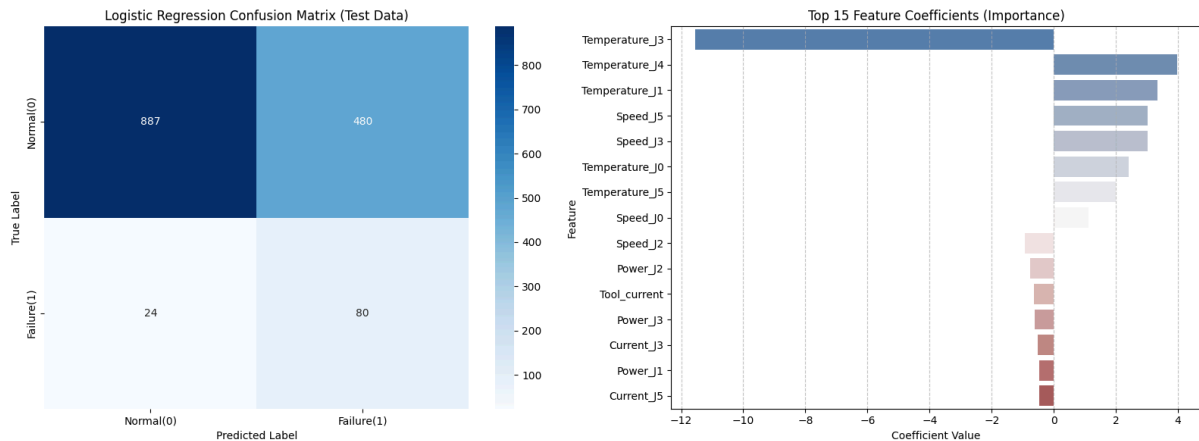
이러한 한계를 극복하고자 **SMOTE**를 통해 생성된 데이터 중, 다른 클래스의 영역에 잘못 침범했거나 경계를 흐리는 노이즈 데이터를 언더샘플링으로 제거해보았다.

즉, 오버샘플링 이후 언더샘플링을 진행했을때의 결과다.

--- [모델 비교] 상위 10개 Logistic Regression 성능 종합 ---									
	순위	CV F1	Train F1	Test F1	과적합 Gap	Train Acc	Test Acc	Test ROC-AUC	파라미터 설정
0	1	0.4872	0.5055	0.5208	-0.0153	0.664854	0.677090	0.772417	C=10, P=I2, S=saga
1	2	0.4861	0.5046	0.5222	-0.0176	0.663834	0.679130	0.772691	C=100, P=I1, S=saga
2	3	0.4860	0.5051	0.5233	-0.0182	0.664174	0.679130	0.772755	C=10, P=I1, S=saga
3	4	0.4860	0.5048	0.5229	-0.0181	0.663664	0.678450	0.772684	C=100, P=I2, S=saga
4	5	0.4853	0.5062	0.5188	-0.0126	0.665874	0.675731	0.772677	C=1, P=I1, S=saga
5	6	0.4797	0.5027	0.5208	-0.0181	0.660945	0.677090	0.769294	C=1, P=I2, S=saga
6	7	0.4668	0.4975	0.5125	-0.0150	0.652447	0.666213	0.769702	C=0.1, P=I1, S=saga
7	8	0.4655	0.4940	0.5099	-0.0159	0.646839	0.665534	0.766438	C=0.1, P=I2, S=saga
8	9	0.4578	0.4919	0.5085	-0.0166	0.646159	0.660095	0.762225	C=0.01, P=I2, S=saga
9	10	0.4553	0.4889	0.5067	-0.0178	0.645309	0.657376	0.745315	C=0.01, P=I1, S=saga

교차검증(CV), Train, Test 모든 구간에서 **F1-Score**와 **Accuracy**가 동반 상승하였다.

다수 클래스 데이터 중 소수 클래스 무리에 너무 깊숙이 침투해 있어 모델을 헛갈리게 만드는 경계선 또는 노이즈 데이터를 삭제하면서 결과적으로 경계선이 깔끔해지면서 모델이 정상과 고장을 구분하는 선을 긋기가 훨씬 수월해졌고 오탐지가 줄어들어 **F1-Score**와 **Accuracy**가 동시에 상승한 것으로 보인다.



Confusion Matrix를 살펴보면 실제 고장인 TP를 2건 더 찾아냈고 놓친 고장인 FP는 2건 줄었다.

하지만 FP는 단 1건도 줄지 않았다. 즉, 앞선 EDA에서 봤듯이 데이터셋은 직선으로는 절대 풀 수 없는 복잡한 비선형 구조를 띄고있음을 증명한다.

언더샘플링을 적용해 경계선을 깔끔히 만들었을때 변수 중요도가 크게 변했다.

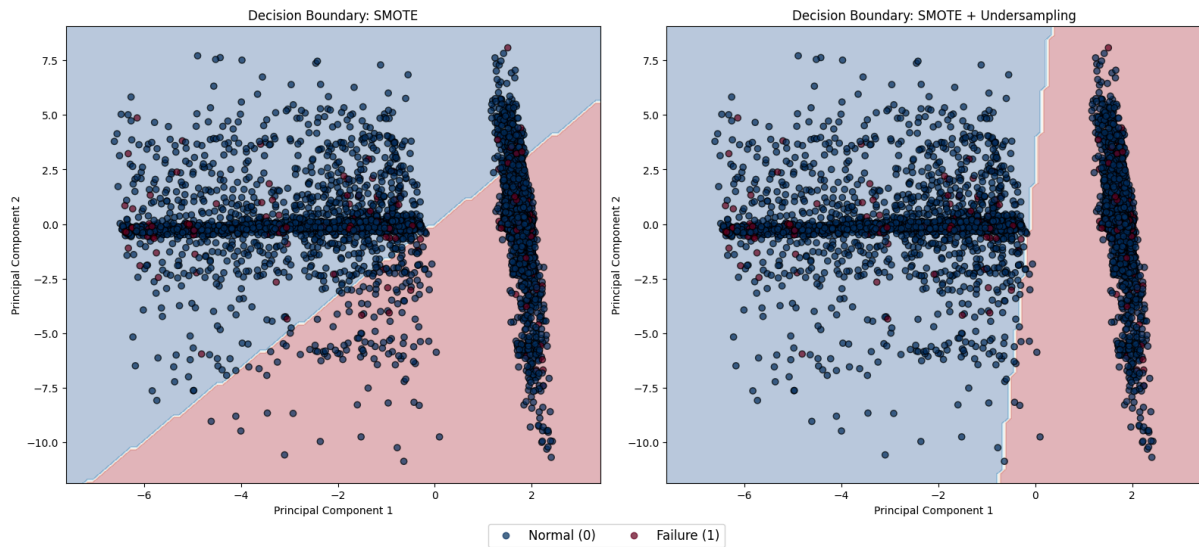
그래프의 X축 스케일을 보면 SMOTE 그래프는 대략 -6 ~ +3 사이였던 반면 언더샘플링을 하고 난 후의 그래프는 -12 ~ +4 이상으로 급격하게 커졌다.

변수들의 순위는 비슷하지만, 모델이 각 변수에 부여하는 확신이 비정상적으로 커졌다.

이유로는 고장과 정상 데이터가 서로 맞닿아 헛갈리는 구역의 데이터를 지워버렸기에 두 클래스 사이에 빈 공간이 생긴다.

따라서 모델 입장에서는 경계선 근처에 방해물이 사라졌으니, 잘 분류해냈다고 착각하게 된다.

이 확신이 수학적으로는 계수의 크기 증가로 나타났다고 추측한다.



26개의 피처를 2차원으로 PCA 한 후 그려본 Decision Boundary를 보면 파란색 점이 정상, 빨간색 점이 고장이다.

빨간 점들은 거대한 파란 점 무리 속에 완벽하게 파묻혀 있다. 따라서 Logistic Regression이 이 데이터에서 절대 높은 성능을 낼 수 없는 근본적인 이유라고 할 수 있다.

왼쪽 그래프는 SMOTE만 적용했을 때 그은 결정 경계 그래프이다. 억지로 늘어난 빨간 점들을 어떻게든 많이 맞추려고 오른쪽 파란 기둥과 왼쪽 파란 뭉치의 하단을 대각선으로 분단해 빨간색 배경 안에 갇혀있는 수많은 파란 점들이 오차 행렬에서 보았던 480개의 False Positive라고 할 수 있다.

오른쪽 그래프는 SMOTE에서 언더샘플링을 적용해 경계선 부분의 데이터를 지우고 그은 결정 경계 그래프이다. 헛갈리는 경계면의 데이터를 지웠더니, 모델이 결정 경계를 대각선에서 수직으로 그었다.

대각선으로 그으나, 수직으로 그으나, 결국 뭉쳐있는 파란 점들을 엄청나게 썰어낼 수밖에 없다. 왼쪽에서 대각선에 잘려 나갔던 파란 점의 개수나, 오른쪽에서 수직선에 갇혀버린

파란 점의 개수가 비슷하게 480여 개였기 때문에, 오차 행렬의 FP 숫자가 단 1건도

줄어들지 않고 똑같았던 것으로 추측할 수 있다.

언더샘플링 이후 특정 변수의 가중치가 갑자기 폭등했던 이유는 모델이 이렇게 판단

기준선의 방향을 완전히 틀어버렸기 때문에, 중요하게 생각하는 변수의 순위도 요동쳤던

것이다.

4. Support Vector Machine

LR의 실패 원인은 직선으로 나눌 수 없는 데이터 구조 때문이라는 것을 잘 확인했다.

SVM은 RBF 커널이라는 방법을 써서 2차원/3차원 데이터를 고차원 데이터로 구부려서

분류한다.

만약 RBF SVM의 성능이 LR보다 확연히 좋고 FP가 줄어든다면, 비선형 모델인 DT나 RF의

성능도 기대해볼 수 있다.

```
df.columns = df.columns.str.strip()
df.rename(columns={'Temperature_T0': 'Temperature_J0'}, inplace=True)

# 2. 결측치 없는 클린 데이터 분리
df_clean = df.dropna().copy()

# 3. 파생 변수 생성 (Power = Current * Speed 등)
current_cols = [f'Current_J{i}' for i in range(6)]
speed_cols = [f'Speed_J{i}' for i in range(6)]
temp_cols = [f'Temperature_J{i}' for i in range(6)]
for i in range(6):
    df_clean[f'Power_J{i}'] = df_clean[f'Current_J{i}'] * df_clean[f'Speed_J{i}']

df_clean['Total_Power_Electric'] = (df_clean[current_cols]**2).sum(axis=1)

# 4. 입력 피쳐(X) 설정
input_cols = current_cols + speed_cols + temp_cols + ['Tool_current'] + [f'Power_J{i}' for i in range(6)] + ['Total_Power_Electric']
X = df_clean[input_cols]

print(f"데이터 크기: {df_clean.shape}")
print(f"피쳐 개수: {len(input_cols)}개")
✓ 0.0s

데이터 크기: (7355, 31)
피쳐 개수: 26개
```

26개의 피쳐를 모두 사용한다.

```

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Imblearn Pipeline 구축
# probability=True: ROC-AUC 계산 및 확률 출력을 위해 필요 (단, 학습 시간이 조금 늘어남)
pipeline = ImbPipeline([
    ('smote', SMOTE(random_state=42)),
    ('svc', SVC(random_state=42, probability=True))
])

# 파라미터 그리드 (비선형 rbf 커널과 선형 linear 커널 비교)
param_grid = {
    'svc__C': [0.1, 1, 10],          # 규제 강도 (오차 허용 범위)
    'svc__gamma': ['scale', 'auto', 0.1], # 결정 경계의 곡률 (rbf 커널용)
    'svc__kernel': ['rbf', 'linear']    # 커널 종류
}

print("--- SVM 하이퍼파라미터 탐색 및 5-Fold 교차 검증 진행 중 (시간이 다소 소요됩니다) ---")
grid_search = GridSearchCV(
    estimator=pipeline,
    param_grid=param_grid,
    cv=skf,
    scoring='f1_macro',
    n_jobs=-1,
    verbose=1
)

grid_search.fit(X_train_scaled, y_train)

print("\n[GridSearchCV 결과]")
print("[최적의 하이퍼파라미터 설정]")
for param, value in grid_search.best_params_.items():
    print(f"    • {param.replace('svc__', '')}: {value}")
print("")
print(f"최고 교차 검증 F1-Macro 점수: {grid_search.best_score_:.4f}")

```

SMOTE를 사용해 클래스 불균형을 해소하고 gridsearch를 진행한다.

C는 규제의 역수, kernel의 종류는 선형 커널인 linear, 저차원에서는 선형 분리가 불가능한

데이터를 고차원 공간으로 매핑하는 RBF를 사용한다.

--- [모델 비교] 상위 10개 SVM 성능 종합 ---

	순위	CV F1	Train F1	Test F1	과적합 Gap	Train Acc	Test Acc	Test ROC-AUC	파라미터 설정
0	1	0.6808	0.7664	0.7258	0.0406	0.911285	0.900748	0.860774	C=10, G=scale, K=rbf
1	2	0.6804	0.7825	0.7112	0.0713	0.917573	0.889871	0.868859	C=10, G=0.1, K=rbf
2	3	0.6776	0.7495	0.7285	0.0210	0.901598	0.899388	0.860883	C=1, G=0.1, K=rbf
3	4	0.6759	0.7526	0.7258	0.0268	0.904657	0.900748	0.863809	C=10, G=auto, K=rbf
4	5	0.6619	0.7088	0.7063	0.0025	0.883923	0.889871	0.863900	C=1, G=scale, K=rbf
5	6	0.6599	0.7004	0.7079	-0.0076	0.878824	0.889871	0.864498	C=1, G=auto, K=rbf
6	7	0.6532	0.6934	0.6929	0.0006	0.877804	0.887831	0.856754	C=0.1, G=0.1, K=rbf
7	8	0.6186	0.6443	0.6676	-0.0233	0.840075	0.859959	0.847497	C=0.1, G=scale, K=rbf
8	9	0.6163	0.6390	0.6676	-0.0287	0.835656	0.858600	0.847645	C=0.1, G=auto, K=rbf
9	10	0.4934	0.5159	0.5341	-0.0181	0.679470	0.690007	0.784783	C=10, G=scale, K=linear

상위 10개의 모델들을 보면, 1위부터 9위까지 RBF 커널을 사용했다. 유일하게 Linear를

사용한 10위 모델은 LR의 결과와 거의 유사하게 나왔다.

나머지 모델들은 F1, Accuracy, ROC-AUC 모두 크게 상승했다.

이전 LR의 최고의 모델과 현재 1위 SVM 모델을 비교하면 F1 score는 0.2 이상 상승했고 약 67%였던 Accuracy가 90% 이상까지 상승했다. 또한 ROC-AUC는 약 0.09가 상승했다.

```
[선택된 3위 모델의 하이퍼파라미터]
• C: 1
• gamma: 0.1
• kernel: rbf

[Train 데이터 평가]
Accuracy (정확도) : 0.9016
F1-Macro Score    : 0.749483
ROC-AUC Score     : 0.932674

[Train 상세 분류 리포트]
| | | | | precision | recall | f1-score | support |
|---|---|---|---|---|
| 0 (Normal) | 0.99 | 0.90 | 0.94 | 5470 |
| 1 (Failure) | 0.41 | 0.87 | 0.55 | 414 |
| | accuracy | | | 0.90 | 5884 |
| | macro avg | 0.70 | 0.89 | 0.75 | 5884 |
| weighted avg | 0.95 | 0.90 | 0.92 | 5884 |

[Test 데이터 평가]
Accuracy (정확도) : 0.8994
F1-Macro Score    : 0.728527
ROC-AUC Score     : 0.860883

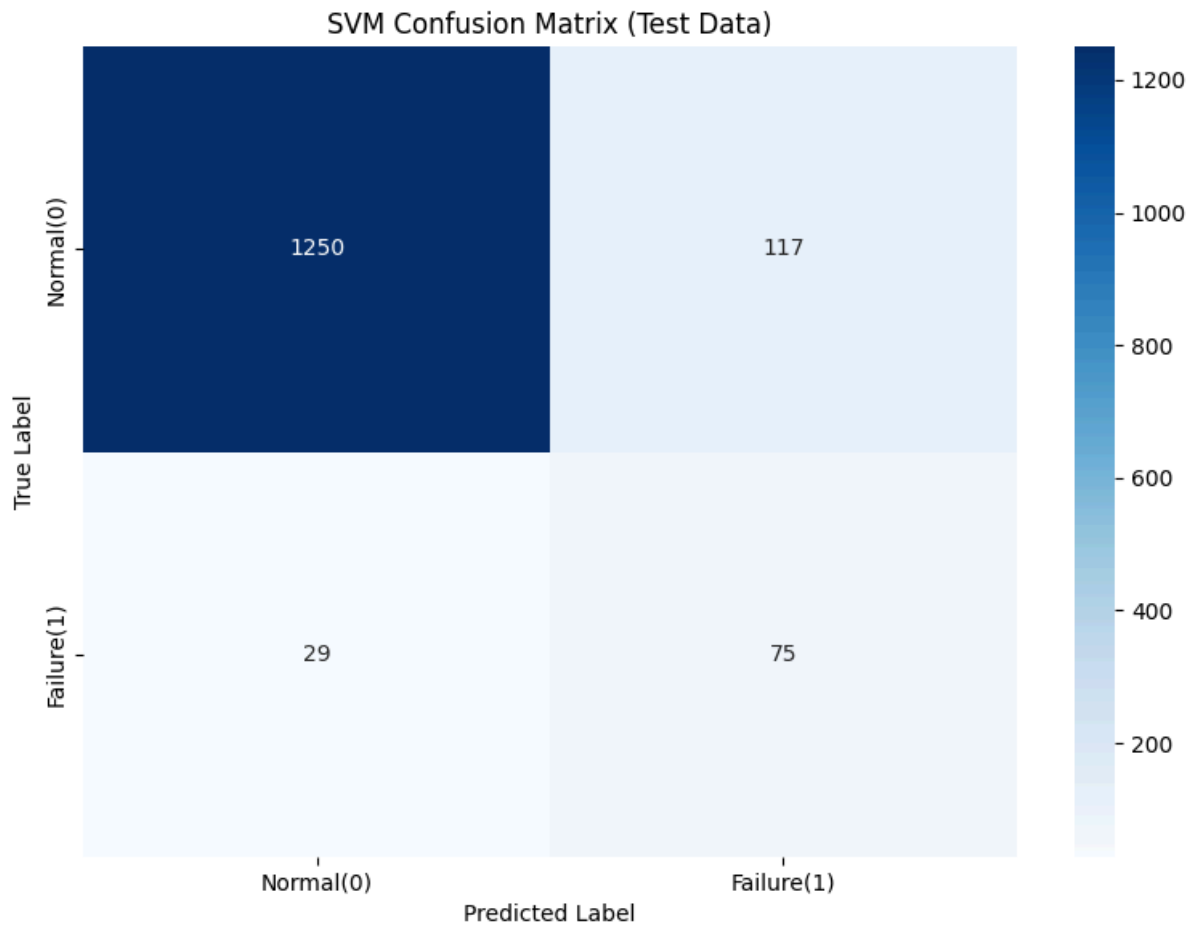
[Test 상세 분류 리포트]
| | | | | precision | recall | f1-score | support |
|---|---|---|---|---|
| 0 (Normal) | 0.98 | 0.91 | 0.94 | 1367 |
| 1 (Failure) | 0.39 | 0.75 | 0.51 | 104 |
| | accuracy | | | 0.90 | 1471 |
| | macro avg | 0.68 | 0.83 | 0.73 | 1471 |
| weighted avg | 0.94 | 0.90 | 0.91 | 1471 |
```

가장 Test F1이 높고 과적합 Gap이 낮은 3위 모델의 Failure 세부 지표를 보면

Precision은 0.39로 전보다 약 2.8배 높아졌다.

Recall은 전과 같은 0.75를 기록했다.

F1 score는 0.51로 2배 이상 높아졌다.



오차 행렬을 보면 **False Positive**가 117건으로 480건이던 LR에 비해 약 **75%**나 줄어들었다.

또한 **True Negative**가 그만큼 증가하여 1250건이 되었다. **Accuracy**가 약 **90%**까지 증가한 이유이다.

True Positive는 75건으로 80건이던 LR에 비해 5건을 놓쳤다. 하지만 **FP**를 363건이나 줄이는 성과를 얻기 위해 **TP**를 5건 양보한것으로 **Precision**이 약 **2.8배** 늘었으니 성공적인 교환비다.